SISTEMAS COMPUTACIONAIS AVANÇADOS (SISTCA)

ADVANCED COMPUTING SYSTEMS

LICENCIATURA EM ENGENHARIA ELECTROTÉCNICA E DE COMPUTADORES

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO
POLITÉCNICO DO PORTO

Lab Classes Script:

Introduction to LiteRT



Version Control

Version number	Date issued	Authors	Update information
V1.0	<março> 2026	Simão Ribeiro (1230692) João Azeredo (1230649) Gilberto Pereira (1221908) João Silva (1221116) Supervision: Prof. Nelson Novais	Original version of the script.

Table of Contents

1. Introduction 1

1.1 Scientific/Technological Context 1

1.2 Motivation for this topic/lab script 1

1.3 Objectives 1

1.4 Structure 2

2. Theoretical (scientific/technological background) 2

2.1 Model Format and Optimization 3

2.2 Converter and Interpreter 3

2.3 Kernels, Quantization, and Acceleration 3

2.4 State-of-The-Art 4

2.4.1 Edge AI and On-Device Inference 4

2.4.2 LiteRT in Embedded and Mobile Systems 4

2.4.3 Quantization for Resource-Constrained Devices 5

2.4.4 Recent Advances in Quantization Research 5

2.4.5 Quantization Accuracy and Performance Trade-Offs 6

2.4.6 Comparison Between Inference Frameworks 7

3. Outlook of the technology 8

3.1 Setup/Installation (SW/HW) 8

3.2 First steps 8

4. Tutorial/Functionality 8

5. Exercise(s) 8

5.1 Problem A 8

5.1.1 (re)solution A 8

5.2 Problem B 8

5.2.1 (re)solution B 8

6. Challenge 8

7. Referências 9

1. Introduction

1.1 Scientific/Technological Context

Artificial intelligence (AI) and machine learning (ML) are two basic technologies that help computers look at large amounts of data and make accurate predictions. These hard computing tasks used to only work on huge cloud servers. But these smart models need to be put directly on edge devices increasingly, like smartphones and Internet of Things (IoT) sensors.

This change makes a big problem worse: edge devices have very little memory, processing power, and battery life. LiteRT is useful in exactly this kind of situation. LiteRT is a lightweight, highly optimized runtime that is made to run machine learning models well on hardware with limited resources. It lets apps do complex, real-time tasks like image recognition and data processing entirely on the device, with no network latency and the highest level of user privacy without needing a cloud connection.

1.2 Motivation for this topic/lab script

The effective implementation of Machine Learning (ML) models has evolved from being a creative add-on to the fundamental component of contemporary computing systems in today's technological environment. The real frontier of artificial intelligence now lies not only in developing extremely complex models but also in making sure these models can function flawlessly, quickly, and unrelentingly in any kind of environment.

The main practical issue is that these models must be deployed on edge devices for a wide range of real-world applications. We are referring to wearables, smartphones, industrial sensors, and Internet of Things devices that have severely limited memory, drastically lower processing power, and severe battery limitations. Because of latency, expenses, and privacy issues, it is not practical to continuously rely on supercomputers or cloud connections in these situations.

Lightweight technologies—carefully crafted with an unwavering focus on efficiency, mathematical optimisation, and incredibly low resource consumption—have become incredibly relevant in the market because of these harsh real-world demands. To get past these physical obstacles and enable artificial intelligence to literally fit in the user's pocket, it is now essential to develop and implement optimised runtime platforms. For anyone hoping to create the autonomous and responsive technological solutions that the future requires, mastering these tools and comprehending their architecture is not only an academic advantage but a necessary skill.

1.3 Objectives

The aim of this document is to provide a clear guide to setting up and initiating LiteRT. It covers basic concepts that are required to understand LiteRT and provides information about how to install and start it. Furthermore, this document provides a practical example of how to load a machine learning model and execute it, which clarifies how to run AI models on edge devices.

1.4 Structure

The remainder of this document is organized to provide a natural progression from theory to complex application.

After this introductory section, the next chapter provides the fundamental theoretical background by establishing the scientific and technological context within which LiteRT is developed. This includes an exploration of fundamental concepts and terms, as well as a brief overview of the state of the art of mobile and embedded machine learning technologies.

Following this section, there is a shift to a more "hands-on" view of the technology. All requisite setup and install procedures for software and hardware components, as well as an overview of the initial steps to get started with the environment. From there, there is a direct progression to a tutorial on the fundamental capabilities of the runtime environment, especially regarding model loading.

The latter part of the script is focused on the practical assessment of these tools. Exercises are included, which are based on problems and their corresponding solutions, to ensure proper knowledge about the implementation process.

This is followed by a final challenge, which helps to check the ability to implement these skills independently. Finally, references and annexes are included for further technical knowledge and troubleshooting.

2. Theoretical (scientific/technological background)

The evolution of machine learning (ML) has seen exponential growth, moving the processing of complex models from cloud servers to edge devices like smartphones and embedded systems. This shift, called on-device ML or edge AI, aims to run model inference - the stage where the model puts the knowledge learned in training to work in real world problems - directly on local hardware. This approach removes the need for constant internet connectivity and centralized servers [1].

LiteRT is Google's next-generation high-performance framework for deploying ML and generative AI on edge devices. It is designed to tackle the challenges of these environments, which have strict limits on memory, CPU cycles, and power use [2] [3]. Unlike traditional training that requires high-performance computing clusters, LiteRT focuses on efficient inference. This allows pre-trained models to run with low latency and better privacy [4].

The primary advantages of this technological approach include:

- **Low Latency:** The elimination of data ”roundtrips” to the cloud enables real-time responsiveness.
- **Privacy:** User data remains on the device, removing the need for transmission to external servers.
- **Connectivity:** Functionality is independent of internet access.
- **Power Consumption:** Specific optimizations reduce computational overhead, preserving battery life in mobile devices [1] [5].

To understand the mechanics of LiteRT, it is essential to understand the following concepts and technical terms that constitute its execution architecture.

2.1 Model Format and Optimization

The initial process involves converting models trained in frameworks such as TensorFlow, PyTorch, or JAX into the optimized .tflite format. A crucial differentiator of this format is the use of FlatBuffers, which allows the model to be mapped directly into memory without a slow "parsing" stage. This results in significantly faster loading times [2] [3].

2.2 Converter and Interpreter

LiteRT operates using two main components that separate the workload into two stages: preparation and execution.

1. **LiteRT Converter (Off-device):** This is a tool used by the developer on a desktop or cloud environment to transform standard models (such as TensorFlow) into the .tflite format. Its primary role is optimization: it removes unnecessary operations for inference (graph stripping) and applies techniques like quantization.

This adapts the model to the strict memory and processing limits of the target hardware. [2]

2. **LiteRT Interpreter (On-device):** This is the execution engine that lives on the final device (e.g., a smartphone or embedded system). Unlike a standard application, the interpreter reads the .tflite file at runtime, maps operations to optimized mathematical functions (kernels), and manages the data flow between tensors.

It is designed to use minimal resources (RAM and CPU) to ensure fast and efficient inference. [2]

2.3 Kernels, Quantization, and Acceleration

The efficiency of the framework relies on three technical pillars:

- **Kernels:** Low-level implementations of mathematical functions optimized for specific hardware architectures [3].
- **Quantization:** A compression technique that reduces the precision of model weights. This drastically reduces model size and increases execution speed with minimal accuracy loss [4].
- **Delegates:** LiteRT supports Hardware Acceleration by redirecting portions of the execution graph to specialized accelerators (GPUs, DSPs, or NPUs). It also implements an automatic fallback mechanism; if a delegate does not support an operation, the Interpreter returns it to the CPU to ensure continuity [2] [4].

2.4 State-of-The-Art

2.4.1 Edge AI and On-Device Inference

The paradigm shift from centralized cloud computing to decentralized edge intelligence arises from the necessity to achieve real-time responsiveness, reduce bandwidth consumption, and improve data privacy. However, the deployment of neural networks on edge platforms also brings forth significant challenges due to the limited computing resources, memory capacities, and energy constraints associated with these platforms.

To overcome these challenges, recent research has been made towards the application of model optimization methods such as quantization, pruning, and knowledge distillation. Moreover, hardware-aware neural architectures have also been proposed. In this direction, TensorFlow Lite (LiteRT), an open-source framework developed by Google, plays a vital role in the deployment of neural networks on edge platforms. The framework enables the deployment of deep learning models by converting and optimizing the models to a compact form. This enables the deployment of deep learning models on a variety of edge platforms, ranging from smartphones to IoT platforms and microcontrollers.

2.4.2 LiteRT in Embedded and Mobile Systems

In the context of edge computing and computer vision, Tunggorono [6] explored the deployment of real-time human detection systems on mass-market, cost-constrained hardware. Specifically, their study utilized an Android TV settop box to demonstrate that low-cost, fanless devices can execute responsive, on-device machine learning without the need for specialized external accelerators. This on-device approach highlights significant practical benefits: it preserves data privacy because no raw video frames leave the premises, reduces network backhaul, and ensures system resilience even when internet connectivity is unreliable. The authors implemented an end-toend pipeline to process video feeds via the Real-Time Streaming Protocol (RTSP). To determine the most effective setup for this hardware, they evaluated several compact object detection models using TensorFlow Lite, testing both CPU and NNAPI (Neural Networks API) runtime delegates. Figure 1 - Qualitative detections from the trained SSD-MobileNet V2 on SISTCA 5/8 The study demonstrated that LiteRT can provide practical real-time performance even on low-cost consumer hardware, particularly when combined with NNAPI acceleration.



Figure 1

2.4.3 Quantization for Resource-Constrained Devices

To bridge the gap between model development and edge deployment, this study [7] relies on a systematic optimization pipeline centered around quantization. After initially training the models offline using the TensorFlow framework, the authors apply post-training quantization to convert the network's parameters from floatingpoint numbers into lower-precision integers. The model's size and computational demands are drastically reduced without suffering substantial accuracy degradation. Furthermore, the study highlights the use of mixed-precision quantization strategies, which allow highly sensitive control parameters to remain at 16-bit precision while less critical calculations are compressed down to 8-bit or 4-bit representations. Once fully optimized, the models are converted into the LiteRT format to leverage its highly efficient runtime environment on the memory-constrained RP2040 microcontroller.

The experimental results show that this compression approach can maintain realtime performance and energy efficiency in autonomous robotics applications. By reducing model size and computational complexity, quantized neural networks were able to run effectively on resource-constrained microcontrollers while still maintaining acceptable navigation accuracy. In addition, the reduced memory footprint and low power consumption make these approaches highly suitable for battery-powered embedded systems.

These results demonstrate that quantization is one of the most important techniques for enabling neural network inference on highly constrained devices.

2.4.4 Recent Advances in Quantization Research

Recent research has also explored more advanced quantization methods. One example is TURBOQUANT [7], a technique designed to reduce memory usage while preserving model quality in large AI systems. Unlike traditional quantization methods, TURBOQUANT achieves high compression rates with only minimal impact on accuracy.

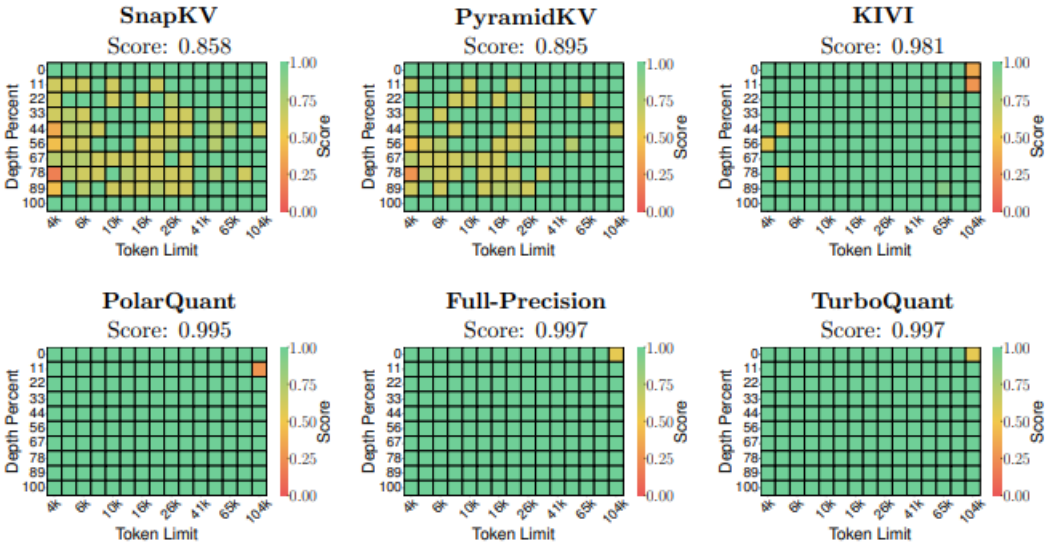


Figure 2 - Comparison of different quantization methods on the "Needle-In-A-Haystack" retrieval test, showing that TURBOQUANT achieves performance close to the full-precision baseline

Experimental results show that the method can preserve the same performance as full-precision models even when the data is compressed by more than 4x. As shown in Figure 4, TURBOQUANT achieved results comparable to the uncompressed baseline in the "Needle-In-A-Haystack" retrieval test, outperforming several existing quantization approaches. These results suggest that advanced quantization methods may play an important role in future edge AI and generative AI applications.

2.4.5 Quantization Accuracy and Performance Trade-Offs

Model's quantization offers very minimal loss of accuracy while reducing needed memory. Various research compare different algorithms and their accuracy:

Features/Model	DNN model		
	DenseNet121	*ResNet50*	*SE-PreAct-ResNet101*
Model Size	7.0M	23.5M	42.5M
Initial Accuracy	95.51%	95.13%	94.76%
PTDRQ	95.47%	95.15%	**94.63%**
PTIQ	**95.08%**	**95.03%**	94.69%
PTFQ	95.51%	95.13%	94.75%
PTIQA	95.52%	95.12%	94.72%

Figure 3 - Comparison of different post-training quantization methods and their impact on model accuracy

Figure 2 presents a comparison between different post-training quantization methods applied to DenseNet121, ResNet50, and SE-PreAct-ResNet101 models. The results show that most quantization techniques preserve accuracy very close to the original full-precision models, with only marginal differences across architectures.

These findings reinforce the idea that quantization is a practical and effective strategy for deploying deep learning models on resource-constrained devices.

2.4.6 Comparison Between Inference Frameworks

The following benchmark compares three widely used on-device inference frameworks — LiteRT, ONNX Runtime, and PyTorch Mobile — using the same image classification model on a Samsung Galaxy S21. The evaluation focuses on inference latency and memory usage to determine which framework is most suitable for mobile deployment.

Metric	LiteRT	ONNX Runtime	PyTorch Mobile
Framework	LiteRT (fmr. TFLite)	ONNX Runtime	PyTorch Mobile
Inference latency (ms)	23 ms	31 ms	38 ms
Memory footprint (MB)	~89 MB	~112 MB	~126 MB
Hardware target	Mobile CPU/GPU/NPU	Cross-platform	Mobile (native PT)
Model format	.tflite (FlatBuffers)	.onnx	TorchScript/.ptl
Multi-framework input	PyTorch, TF, JAX	PyTorch, TF, sklearn	PyTorch only
Relative latency vs best	1.00x (baseline)	1.35x slower	1.65x slower

LiteRT delivered the best overall performance, achieving both the lowest inference latency and the smallest memory footprint. These characteristics make it particularly suitable for mobile and edge devices with limited computational resources. ONNX Runtime provides a good balance between performance and cross-platform compatibility, while PyTorch Mobile remains a practical option for teams already working within the PyTorch ecosystem [8].

3. Outlook of the technology

(provide a snapshot of the main characteristics/functionality of the technology described in this lab script)

3.1 Setup/Installation (SW/HW)

(describe the way to start using the technology, namely the setpup/installation SW/HW requirements, operating systems, development platforms,…)

3.2 First steps

(probably you can fit here something like the basic steps for a outputting a "hello world" message (broadly speaking) ☺)

4. Tutorial/Functionality

(this section is the core of the lab script, where all the "action" will happen; the student should be guided through a step-by-step execution, with supporting help/explanation of each functionality, towards a final objective, such as building a 3D model of a boat or producing a prototype hardware board)

5. Exercise(s)

(this section may include one or more illustrative exercises, providing the corresponding resolution)

5.1 Problem A

5.1.1 (re)solution A

5.2Problem B

5.2.1 (re)solution B

…

6. Challenge

(this section challenges students to do something different, to solve a specific problem, using the core technology/tool of this lab script; some hints may be provided, but not the (re)solution, which may be unveiled afterwards (e.g. one week later) or upon explicit solicitation from students that unsuccessfully attempted to solve the problem)

7. Referências

[1] V. S. R. Narapareddy e S. K. Yerramilli, "Edge AI and On-Device Machine Learning," Zenodo, 2025.

[2] G. A. f. Developers, "LiteRT: Framework for On-Device ML," 2026. [Online]. Available: https://ai.google.dev/edge/litert. [Acedido em 25 03 2026].

[3] R. David e e. al., "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," arXiv, 2020.

[4] R. Immonen e T. Hämäläinen, "Tiny Machine Learning for Resource-Constrained Microcontrollers," *Journal of Sensors,* pp. 1-11, 2022.

[5] Y. Deng, "Deep Learning on Mobile Devices - A Review," arXiv, 2019.

[6] N. Dashdamirli, " Application of Neural Networks for Robotics Control on the RP2040 Microcontroller Using LiteRT. Engineering mechanics.," 2025.

[7] Zandieh, "TurboQuant: Online Vector Quantization with Near-optimal," 2026.

[8] I. L. Orăşan, C. Seiculescu e C. D. Caleanu, "Benchmarking TensorFlow Lite Quantization Algorithms for Deep Neural Networks," 2022.

[9] Z. M. Tunggorono, R. R. Hariadi e A. Bramantoro, "Real-Time Human Detection on an Amlogic S905X Android Set-Top Box Using TensorFlow Lite," 2025.

[10] "Optimize Production with PyTorch/TF, ONNX, TensorRT & LiteRT," DigitalOcean, 3 Outubro 2025. [Online]. Available: https://www.digitalocean.com/community/tutorials/ai-model-deployment-optimization. [Acedido em 1 Abril 2026].

Annexes

(here you should include any technical documentation that you consider relevant for supporting the lab script but does not "fit into", as you feel it would jeopardize the reading flow of the core text)